

Chương 1

Từ Monolithic đến Microservice và DDD

1.1 Kiến trúc đơn khối (Monolithic Architecture)

1.1.1 Khái niệm và Bản chất Kỹ thuật

Kiến trúc Monolithic không chỉ đơn thuần là việc đóng gói mã nguồn vào một tệp tin thực thi (như .war hay .jar), mà còn đại diện cho một tư duy thiết kế tập trung. Trong mô hình này, toàn bộ logic nghiệp vụ, giao diện người dùng, và các thành phần truy xuất dữ liệu được liên kết chặt chẽ (tightly coupled) trong một đơn vị duy nhất.

Sự gắn kết này có nghĩa là các thành phần chia sẻ chung bộ nhớ, tài nguyên CPU và thường là một cơ sở dữ liệu quan hệ duy nhất. Bất kỳ thay đổi nhỏ nào trong một module cũng đòi hỏi việc biên dịch lại và triển khai lại toàn bộ hệ thống.

1.1.2 Cấu trúc Phân lớp điển hình

Đa số các hệ thống Monolithic hiện đại được tổ chức theo mô hình ****Layered Architecture**** (Kiến trúc phân lớp):

- **Presentation Layer:** Xử lý giao diện người dùng và các yêu cầu HTTP.
- **Business Logic Layer:** Chứa các quy tắc nghiệp vụ cốt lõi của hệ thống.
- **Data Access Layer (DAL):** Chịu trách nhiệm giao tiếp với cơ sở dữ liệu.

Mặc dù được phân lớp về mặt logic, nhưng ở cấp độ vật lý, chúng vẫn chạy chung một tiến trình (process).

1.1.3 Ưu điểm và Hạn chế trong Vận hành

Ưu điểm:

- **Đơn giản trong quản lý:** Quá trình kiểm thử tích hợp (End-to-End testing) diễn ra thuận lợi do không có độ trễ mạng giữa các thành phần.
- **Hiệu suất tối ưu:** Các lời gọi hàm trực tiếp trong bộ nhớ luôn nhanh hơn việc gọi qua API hoặc Message Broker.
- **Nhất quán dữ liệu (ACID):** Việc sử dụng một DB duy nhất giúp đảm bảo tính toàn vẹn dữ liệu thông qua các Transaction truyền thống.

Hạn chế (The Big Ball of Mud):

- **Rào cản về quy mô (Scalability):** Không thể mở rộng riêng biệt một module bị nghẽn (ví dụ: module xử lý thanh toán), buộc phải nhân bản toàn bộ ứng dụng, gây lãng phí tài nguyên.
- **Sự trì trệ về công nghệ (Technology Lock-in):** Việc nâng cấp một thư viện hoặc ngôn ngữ mới là cực kỳ rủi ro vì nó ảnh hưởng đến tất cả các phần còn lại.

1.2 Kiến trúc Dịch vụ nhỏ (Microservices Architecture)

1.2.1 Triết lý thiết kế và Đặc điểm cốt lõi

Khác với Monolithic, Microservices tuân thủ triết lý "Làm một việc và làm thật tốt" (Do one thing and do it well). Mỗi dịch vụ là một thực thể độc lập, có vòng đời phát triển và triển khai riêng biệt.

Đặc điểm quan trọng nhất là **Bao đóng dữ liệu (Data Encapsulation)**: Mỗi microservice sở hữu cơ sở dữ liệu riêng (Database per Service), ngăn chặn việc truy cập dữ liệu trực tiếp từ các dịch vụ khác, từ đó giảm thiểu sự phụ thuộc.

1.2.2 Hệ sinh thái và Các thành phần hỗ trợ

Để một hệ thống Microservices vận hành ổn định, cần có các thành phần hạ tầng phức tạp:

- **API Gateway:** Điểm tiếp nhận duy nhất cho client, chịu trách nhiệm định tuyến, xác thực và giới hạn lưu lượng.
- **Service Discovery:** Cho phép các dịch vụ tìm thấy nhau trong môi trường mạng động (ví dụ: Netflix Eureka, Consul).
- **Circuit Breaker:** Cơ chế ngăn chặn lỗi dây chuyền khi một dịch vụ gặp sự cố (ví dụ: Hystrix hoặc Resilience4j).

1.2.3 Thách thức về Quản trị và Nhất quán dữ liệu

Mặc dù mang lại sự linh hoạt, Microservices đối mặt với bài toán **Nhất quán sau cùng (Eventual Consistency)**. Do dữ liệu bị phân tán, việc thực hiện các giao dịch phân tán (Distributed Transactions) trở nên cực kỳ khó khăn, thường phải áp dụng các mô hình như **Saga Pattern** để điều phối.

1.3 Phân tích so sánh: Khi nào nên chuyển đổi?

Việc lựa chọn kiến trúc không dựa trên xu hướng, mà dựa trên "Complexity-Value Curve" (Đường cong giá trị và độ phức tạp).

- Với các dự án khởi nghiệp cần MVP nhanh, Monolithic là sự lựa chọn tối ưu.
- Khi số lượng lập trình viên tăng lên (trên 20-30 người) và các yêu cầu nghiệp vụ trở nên quá chằng chéo, việc phân rã sang Microservices trở thành yếu tố sống còn để duy trì tốc độ phát triển.

1.4 Thiết kế hướng tên miền (Domain-Driven Design - DDD)

1.4.1 Tại sao DDD là chìa khóa cho Microservices?

Một trong những lỗi lớn nhất khi triển khai Microservices là phân rã dịch vụ theo kỹ thuật (ví dụ: Service DB, Service UI). DDD giải quyết vấn đề này bằng cách tập trung vào nghiệp vụ cốt lõi.

1.4.2 Chiến lược thiết kế (Strategic Design)

Khái niệm quan trọng nhất là **Bounded Context** (Ngữ cảnh bao đóng). Nó xác định ranh giới logic mà trong đó một mô hình dữ liệu có ý nghĩa nhất định. Ví dụ: Từ "Sản phẩm" trong ngữ cảnh "Bán hàng" sẽ khác với "Sản phẩm" trong ngữ cảnh "Kho bãi". Việc xác định đúng các Bounded Context chính là bước đầu tiên để xác định ranh giới của các Microservices.

1.4.3 Context Mapping: Bản đồ tương tác

Context Map giúp mô tả mối quan hệ giữa các ngữ cảnh:

- **Shared Kernel:** Hai ngữ cảnh chia sẻ chung một phần mô hình.
- **Anticorruption Layer (ACL):** Lớp đệm giúp dịch vụ mới không bị ảnh hưởng bởi thiết kế lỗi thời của hệ thống cũ.

1.4.4 Thiết kế kỹ thuật (Tactical Design)

Để thực thi logic bên trong một Microservice, DDD cung cấp các công cụ:

- **Aggregate:** Một cụm các đối tượng dữ liệu được coi là một đơn vị để thay đổi dữ liệu.
- **Value Objects:** Các đối tượng không có định danh, chỉ chứa giá trị (như Địa chỉ, Tiền tệ).
- **Repositories:** Lớp trung gian giữa Domain và Data Mapping.

1.5 Case Study: Healthcare (Luyện Decomposition)

1.5.1 Mô tả bài toán

Để củng cố lý thuyết về DDD và quy trình phân rã Microservices, chúng ta sẽ phân tích một hệ thống quản lý bệnh viện (Hospital Management System - HMS). Mục tiêu là chuyển đổi một hệ thống quản lý bệnh viện từ kiến trúc Monolithic sang Microservices để giải quyết các vấn đề về hiệu suất và khả năng mở rộng khi số lượng bệnh nhân và dịch vụ y tế tăng cao.

1.5.2 Thiết kế hệ thống

Mô tả bài toán chi tiết

Hệ thống quản lý bệnh viện cốt lõi tập trung vào việc số hóa các quy trình vận hành lâm sàng, bao gồm:

- **Quản lý bệnh nhân:** Tiếp nhận, lưu trữ thông tin cá nhân và quản lý hồ sơ bệnh án điện tử.
- **Quản lý bác sĩ:** Quản lý danh sách nhân sự y tế, chuyên khoa và lịch trực.
- **Đặt lịch khám:** Tiếp nhận yêu cầu đặt lịch từ bệnh nhân, điều phối phòng khám và thời gian của bác sĩ.

Bước 1: Xác định Domain

Trong hệ thống HMS, các Domain nghiệp vụ chính được xác định bao gồm:

- **Patient Management Domain:** Chịu trách nhiệm về mọi dữ liệu liên quan đến người bệnh.
- **Doctor Management Domain:** Chịu trách nhiệm về nguồn lực y tế.
- **Appointment Scheduling Domain:** Logic điều phối trung tâm của bệnh viện.
- **Billing & Insurance Domain:** Quản lý tài chính và bảo hiểm y tế.

Bước 2: Xác định Bounded Context

Áp dụng chiến lược thiết kế DDD, chúng ta phân tách các ngữ cảnh bao đóng để xác định ranh giới mô hình:

- **Patient Context:** Tập trung vào thực thể Bệnh nhân và quy trình đăng ký.
- **Doctor Context:** Quản lý trạng thái sẵn sàng và thông tin chuyên môn của Bác sĩ.
- **Appointment Context:** Tập trung vào thực thể Lịch hẹn, nơi kết nối giữa Bệnh nhân và Bác sĩ trong một khung giờ cụ thể.

Bước 3: Phân rã thành Microservices

Dựa trên các Bounded Context, hệ thống được phân rã thành các dịch vụ độc lập về mặt vật lý:

- **Patient Service:** Quản lý CRUD thông tin bệnh nhân và hồ sơ cơ bản.
- **Doctor Service:** Cung cấp thông tin bác sĩ theo chuyên khoa cho các dịch vụ khác.
- **Appointment Service:** Chứa logic nghiệp vụ phức tạp về việc kiểm tra xung đột lịch và đặt chỗ.
- **Notification Service:** Dịch vụ phụ trợ gửi thông báo nhắc lịch tự động.

Bước 4: Xác định quan hệ

Mối quan hệ giữa các dịch vụ được thiết lập như sau:

- Dịch vụ **Appointment** phụ thuộc vào **Patient Service** (để lấy thông tin người đặt) và **Doctor Service** (để xác nhận bác sĩ tiếp nhận).
- Giao tiếp giữa các dịch vụ được thực hiện chủ yếu qua **REST API** cho các yêu cầu truy vấn thông tin và **Event-driven** (Message Broker) để cập nhật trạng thái lịch khám.

Giao diện lập trình ứng dụng (API)

Một số API then chốt trong thiết kế:

- POST /appointments/book: Tiếp nhận yêu cầu đặt lịch mới.
- GET /patients/{id}/records: Truy xuất lịch sử khám bệnh của một bệnh nhân.
- GET /doctors/availability: Kiểm tra danh sách bác sĩ có thể tiếp nhận bệnh nhân.

1.5.3 Kết luận

Thông qua Case Study về Healthcare, chúng ta thấy rằng việc phân rã dựa trên DDD giúp mỗi Microservice tập trung hoàn toàn vào một ngữ cảnh nghiệp vụ duy nhất. Sự độc lập này cho phép đội ngũ phát triển có thể nâng cấp thuật toán đặt lịch trong Appointment Service mà không cần can thiệp hay gây gián đoạn cho Patient Service. Đây chính là giá trị cốt lõi của kiến trúc Microservices trong việc duy trì tính bền vững của các hệ thống phức tạp.

Chương 2

Phát triển Hệ thống E-Commerce dựa trên Microservices

2.1 Xác định yêu cầu hệ thống

Trong bối cảnh thị trường thương mại điện tử cạnh tranh khốc liệt, việc xây dựng một hệ thống có khả năng thích ứng nhanh và quy mô lớn là điều bắt buộc. Hệ thống được thiết kế để phục vụ cả khách hàng (Customer) và nhân viên quản trị (Staff), tích hợp trí tuệ nhân tạo (AI) để tối ưu hóa trải nghiệm người dùng.

2.1.1 Yêu cầu chức năng (Functional Requirements)

Hệ thống bao gồm các nhóm chức năng chính sau:

- **Quản lý người dùng và xác thực:** Cho phép khách hàng đăng ký, đăng nhập và quản lý thông tin cá nhân. Nhân viên có các quyền hạn riêng biệt theo vai trò (Role-based Access Control).
- **Quản lý danh mục sản phẩm:** Hiển thị và quản lý thông tin chi tiết các dòng Laptop và Mobile, bao gồm cấu hình kỹ thuật, giá cả và chương trình khuyến mãi.
- **Hệ thống giỏ hàng:** Cho phép người dùng thêm, bớt sản phẩm từ các dịch vụ khác nhau (Laptop/Mobile) vào một giỏ hàng nhất nhất quán.
- **Tư vấn thông minh (AI Chatbot):** Cung cấp khả năng trả lời tự động các thắc mắc về sản phẩm dựa trên cơ sở tri thức (Knowledge Base) được trích xuất từ cơ sở dữ liệu đồ thị.

2.1.2 Yêu cầu phi chức năng (Non-functional Requirements)

Để đảm bảo chất lượng dịch vụ cấp doanh nghiệp, hệ thống cần đáp ứng:

- **Tính mở rộng (Scalability):** Khả năng mở rộng độc lập các dịch vụ có tải cao như Laptop Service hoặc AI Service mà không ảnh hưởng đến toàn bộ hệ thống.
- **Tính sẵn sàng cao (High Availability):** Hệ thống phải hoạt động liên tục, giảm thiểu điểm chết duy nhất (Single Point of Failure) thông qua cơ chế replication và load balancing.
- **Tính bảo mật (Security):** Dữ liệu người dùng phải được mã hóa, các API được bảo vệ qua Gateway và cơ chế xác thực tập trung.
- **Khả năng bảo trì (Maintainability):** Mã nguồn được tổ chức theo cấu trúc chuẩn của Django và Microservices, giúp dễ dàng nâng cấp và sửa lỗi từng phần.

2.2 Phân rã hệ thống theo thiết kế hướng tên miền (DDD)

Việc phân rã hệ thống dựa trên nghiệp vụ thay vì kỹ thuật giúp đảm bảo tính bao đóng và giảm thiểu sự phụ thuộc chéo.

2.2.1 Xác định các Ngữ cảnh bao đóng (Bounded Contexts)

Dựa trên phân tích nghiệp vụ, hệ thống được chia thành 5 Bounded Contexts chính:

- **Customer Context:** Tập trung vào thực thể khách hàng và quy trình tương tác của họ với hệ thống (Login, Register, Cart).
- **Laptop Context:** Quản lý chuyên sâu về các sản phẩm máy tính xách tay với các đặc tính kỹ thuật riêng biệt (CPU, GPU, Screen).
- **Mobile Context:** Quản lý các thiết bị di động với các thuộc tính như Camera, dung lượng Pin.
- **Staff Context:** Quản lý nhân sự và các hoạt động quản trị nội bộ.
- **AI Context:** Cung cấp các khả năng thông minh, xử lý ngôn ngữ tự nhiên để tư vấn bán hàng.

2.2.2 Các nguyên tắc và ràng buộc kiến trúc

Hệ thống tuân thủ các ràng buộc nghiêm ngặt:

- **Database per Service:** Mỗi ngữ cảnh sở hữu một cơ sở dữ liệu riêng, đảm bảo tính cô lập hoàn toàn về dữ liệu.
- **Polyglot Persistence:** Sử dụng đa dạng các loại cơ sở dữ liệu (PostgreSQL cho dữ liệu quan hệ và Neo4j cho dữ liệu đồ thị phục vụ AI).
- **Giao tiếp qua API/Gateway:** Các dịch vụ không gọi trực tiếp cơ sở dữ liệu của nhau mà phải thông qua các giao diện lập trình ứng dụng (API).

2.3 Thiết kế và Triển khai Customer Service

Customer Service đóng vai trò là điểm chạm đầu tiên của người dùng, xử lý các nghiệp vụ liên quan đến định danh và quản lý trạng thái mua hàng.

2.3.1 Logic nghiệp vụ

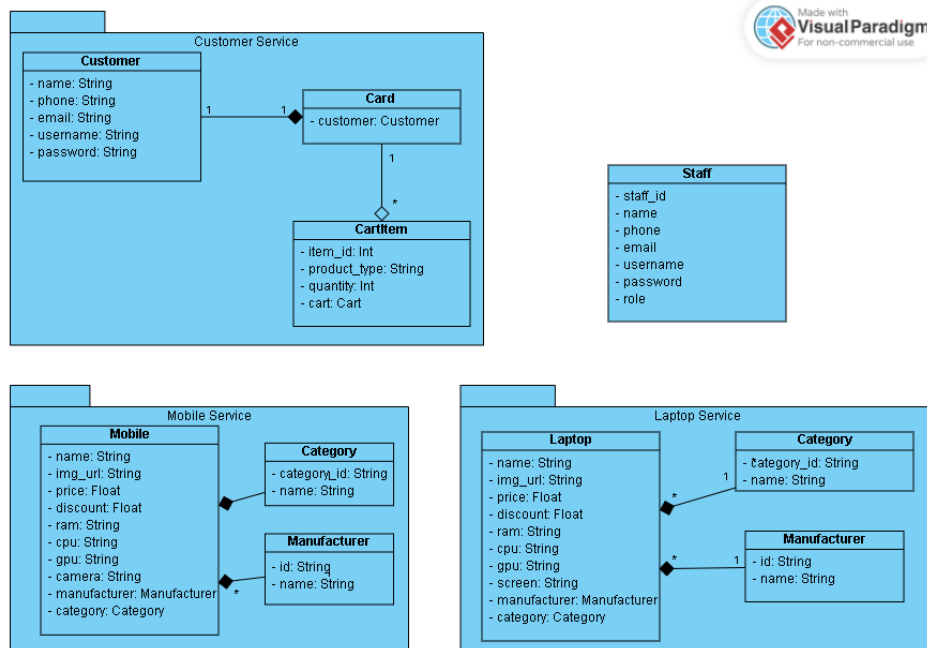
- **Quản lý danh tính:** Xử lý quy trình đăng ký tài khoản mới và xác thực người dùng dựa trên thông tin định danh duy nhất.
- **Quản lý giỏ hàng tập trung:** Khởi tạo giỏ hàng riêng biệt cho mỗi khách hàng ngay khi đăng ký.
- **Xử lý vật phẩm đa dạng:** Cho phép thêm các loại sản phẩm khác nhau (Laptop, Mobile) vào chung một giỏ hàng bằng cách lưu trữ mã định danh và loại sản phẩm.

- **Cập nhật số lượng thông minh:** Tự động kiểm tra sự tồn tại của vật phẩm trong giỏ; nếu đã tồn tại, hệ thống sẽ cộng dồn số lượng thay vì tạo bản ghi mới.

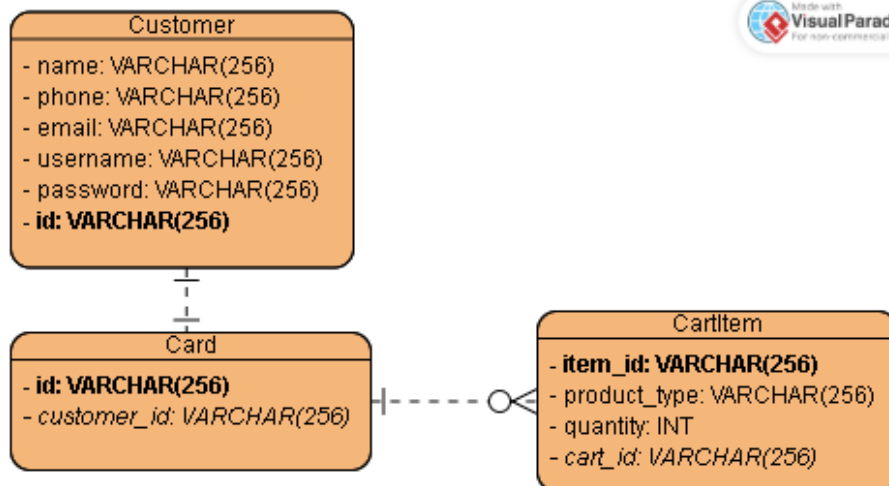
2.3.2 Mô hình dữ liệu (Data Models)

Hệ thống sử dụng các thực thể chính sau:

- **Customer:**
 - name: Họ và tên khách hàng (String).
 - phone: Số điện thoại liên lạc (String).
 - email: Địa chỉ thư điện tử (Unique Email).
 - username: Tên đăng nhập (Unique String).
 - password: Mật khẩu đã được băm (Hash String).
- **Cart:**
 - customer: Liên kết 1-1 với thực thể Customer.
- **CartItem:**
 - item_id: ID của sản phẩm từ dịch vụ ngoại lai (Integer).
 - product_type: Loại sản phẩm (MOBILE hoặc LAPTOP).
 - quantity: Số lượng sản phẩm trong giỏ (Positive Integer).
 - cart: Liên kết N-1 với thực thể Cart.



Hình 2.1: Sơ đồ lớp (Class Diagram)



Hình 2.2: Thiết kế Cơ sở dữ liệu Customer Service

2.3.3 Thiết kế CSDL

2.3.4 Xây dựng cơ sở dữ liệu

Cơ sở dữ liệu sử dụng: MySQL.

Lý do lựa chọn: MySQL là hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) phổ biến, mã nguồn mở, có hiệu suất cao cho các tác vụ đọc/ghi thông tin người dùng và giỏ hàng. Nó đảm bảo tính nhất quán dữ liệu (ACID) cần thiết cho các giao dịch liên quan đến khách hàng.

Mã nguồn định nghĩa các bảng (Django Models):

```

class Customer(models.Model):
    name = models.CharField(max_length=255)
    phone = models.CharField(max_length=20)
    email = models.EmailField(unique=True)
    username = models.CharField(max_length=150, unique=True)
    password = models.CharField(max_length=255)

class Cart(models.Model):
    customer = models.OneToOneField(Customer, on_delete=models.CASCADE)

class CartItem(models.Model):
    item_id = models.IntegerField()
    cart = models.ForeignKey(Cart, on_delete=models.CASCADE)
    quantity = models.PositiveIntegerField(default=1)
    product_type = models.CharField(max_length=10,
                                     choices=(('MOBILE', 'MOBILE'), ('LAPTOP', 'LAPTOP'))
  
```

Phương pháp Seed dữ liệu: Hệ thống sử dụng Django Management Command để khởi tạo dữ liệu mẫu. Lệnh thực hiện:

```
python manage.py seed_db
```

Dữ liệu được tạo bao gồm các tài khoản khách hàng mặc định và giỏ hàng trống tương ứng để sẵn sàng cho việc mua sắm.

2.3.5 Giao diện lập trình ứng dụng (API Endpoints)

Endpoint	Method	Tác dụng	Dữ liệu yêu cầu	Dữ liệu trả về
/register/	POST	Đăng ký tài khoản	Thông tin Customer	JSON khách hàng
/login/	POST	Xác thực người dùng	Username, Password	Thông tin khách hàng
/cart-items/	GET	Liệt kê vật phẩm	Token xác thực	List CartItems
/cart-items/	POST	Thêm vào giỏ	item_id, type, qty	CartItem chi tiết

2.4 Thiết kế và Triển khai Laptop Service

Dịch vụ quản lý kho dữ liệu và đặc tính kỹ thuật của các dòng máy tính xách tay.

2.4.1 Logic nghiệp vụ

- **Phân loại sản phẩm:** Tổ chức máy tính theo hãng sản xuất và danh mục sử dụng (Gaming, Văn phòng, v.v.).
- **Quản lý cấu hình chi tiết:** Lưu trữ các thông số phần cứng chuyên sâu phục vụ việc so sánh và tra cứu.
- **Đồng bộ hóa dữ liệu đồ thị:** Cung cấp dữ liệu để ánh xạ sang Neo4j, xây dựng mạng lưới liên kết giữa sản phẩm và các thành phần phần cứng.

2.4.2 Mô hình dữ liệu (Data Models)

- **Mô hình quan hệ (Relational Model):**
 - **Laptop:** name, img_url, price, discount, ram, cpu, gpu, screen.
 - **Manufacturer / Category:** Chỉ bao gồm trường name để định danh.
- **Mô hình đồ thị (Graph Model):**
 - **Nodes:** Laptop, CPU, GPU, RAM, Screen, Manufacturer.
 - **Relationships:** HAS_CPU, MADE_BY, BELONGS_TO.

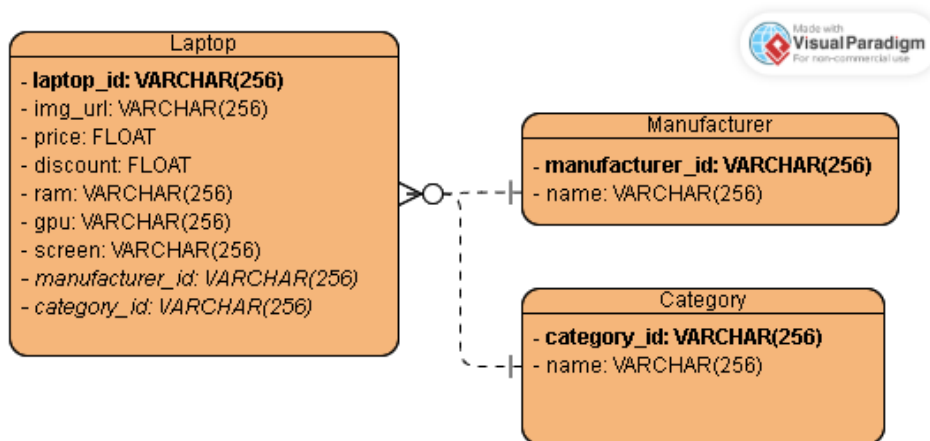
2.4.3 Thiết kế CSDL

2.4.4 Xây dựng cơ sở dữ liệu

Cơ sở dữ liệu sử dụng: PostgreSQL (Relational) và Neo4j (Graph).

Lý do lựa chọn:

- **PostgreSQL:** Được sử dụng để lưu trữ dữ liệu sản phẩm có cấu trúc chặt chẽ. PostgreSQL hỗ trợ tốt các kiểu dữ liệu phức tạp và khả năng mở rộng mạnh mẽ cho danh mục sản phẩm lớn.



Hình 2.3: Thiết kế Cơ sở dữ liệu Laptop Service

- **Neo4j:** Được sử dụng để lưu trữ các mối quan hệ kỹ thuật giữa Laptop và phần cứng (CPU, GPU, RAM). Điều này cho phép AI Service truy vấn và so sánh cấu hình một cách nhanh chóng thông qua các thuật toán đồ thị.

Mã nguồn định nghĩa các bảng (Django Models):

```
class Manufacturer(models.Model):
    name = models.CharField(max_length=255)

class Category(models.Model):
    name = models.CharField(max_length=255)

class Laptop(models.Model):
    name = models.CharField(max_length=255)
    img_url = models.URLField(max_length=500)
    price = models.DecimalField(max_digits=12, decimal_places=2)
    manufacturer = models.ForeignKey(Manufacturer, on_delete=models.CASCADE)
    category = models.ForeignKey(Category, on_delete=models.CASCADE)
    ram = models.CharField(max_length=50)
    cpu = models.CharField(max_length=255)
    gpu = models.CharField(max_length=255)
    screen = models.CharField(max_length=255)
```

Phương pháp Seed dữ liệu: Dữ liệu được khởi tạo thông qua script Python độc lập. Lệnh thực hiện:

```
python seed_db.py
```

Script này sẽ tự động tạo các bản ghi cho Hãng sản xuất, Danh mục và danh sách các Laptop mẫu cùng thông số kỹ thuật chi tiết.

Endpoint	Method	Tác dụng	Dữ liệu yêu cầu	Dữ liệu trả về
/laptops/	GET	Lấy danh sách sản phẩm	Tham số lọc (tùy chọn)	Danh sách Laptop
/laptops/{id}/	GET	Chi tiết sản phẩm	ID sản phẩm	Đối tượng Laptop
/manufacturers/	GET	Lấy danh sách hãng	Không	Danh sách hãng

2.4.5 Giao diện lập trình ứng dụng (API Endpoints)

2.5 Thiết kế và Triển khai Mobile Service

Quản lý dải sản phẩm thiết bị di động và các đặc thù công nghệ liên quan.

2.5.1 Logic nghiệp vụ

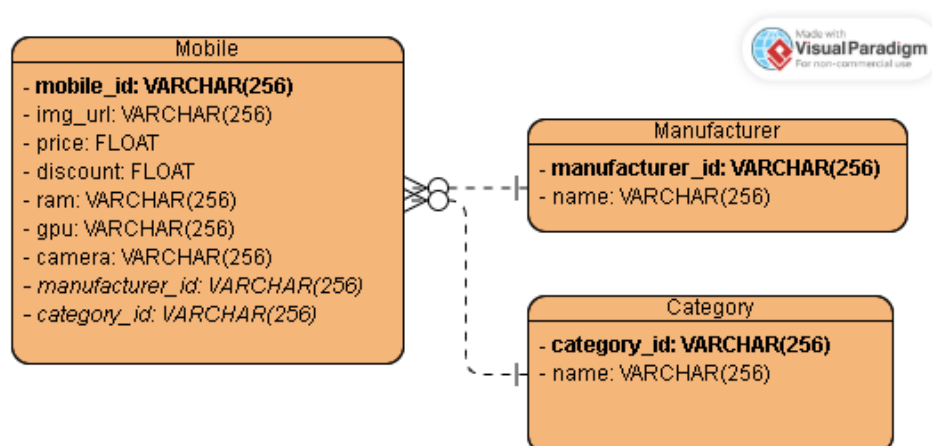
- **Quản lý thông số di động:** Tập trung vào các yếu tố người dùng quan tâm như Camera và hiệu năng xử lý.
- **Phân mảnh thị trường:** Hỗ trợ phân loại sản phẩm theo thương hiệu để người dùng dễ dàng tiếp cận.

2.5.2 Mô hình dữ liệu (Data Models)

• Mobile:

- name, price, discount, camera: Thuộc tính cơ bản và đặc thù.
- ram, cpu, gpu: Cấu hình hiệu năng.
- manufacturer, category: Tham chiếu thực thể cha.

2.5.3 Thiết kế CSDL



Hình 2.4: Thiết kế Cơ sở dữ liệu Mobile Service

2.5.4 Xây dựng cơ sở dữ liệu

Cơ sở dữ liệu sử dụng: PostgreSQL và Neo4j.

Lý do lựa chọn: Tương tự như Laptop Service, Mobile Service sử dụng PostgreSQL cho các thuộc tính thương mại và Neo4j cho các mối quan hệ kỹ thuật. Sự kết hợp này giúp hệ thống vừa đảm bảo tính nhất quán giao dịch, vừa tối ưu cho các tính năng thông minh của AI Chatbot khi cần so sánh các thông số như Camera hay dung lượng Pin giữa các dòng điện thoại.

Mã nguồn định nghĩa các bảng (Django Models):

```
class Manufacturer(models.Model):
    name = models.CharField(max_length=255)

class Category(models.Model):
    name = models.CharField(max_length=255)

class Mobile(models.Model):
    name = models.CharField(max_length=255)
    img_url = models.URLField(max_length=500)
    price = models.DecimalField(max_digits=12, decimal_places=2)
    manufacturer = models.ForeignKey(Manufacturer, on_delete=models.CASCADE)
    category = models.ForeignKey(Category, on_delete=models.CASCADE)
    ram = models.CharField(max_length=50)
    cpu = models.CharField(max_length=255)
    gpu = models.CharField(max_length=255)
    camera = models.CharField(max_length=255)
```

Phương pháp Seed dữ liệu: Sử dụng Django Management Command để khởi tạo danh mục điện thoại. Lệnh thực hiện:

```
python manage.py seed_db
```

Dữ liệu mẫu bao gồm các dòng Flagship phổ biến như iPhone, Samsung Galaxy, Xiaomi,... cùng các thông số phần cứng tương ứng.

2.5.5 Giao diện lập trình ứng dụng (API Endpoints)

Endpoint	Method	Tác dụng	Dữ liệu yêu cầu	Dữ liệu trả về
/mobiles/	GET	Danh sách điện thoại	Không	List Mobile
/mobiles/{id}/	GET	Xem chi tiết	ID	Chi tiết Mobile

2.6 Thiết kế và Triển khai Staff Service

Hệ thống quản trị dành cho nhân sự vận hành sản phẩm thương mại điện tử.

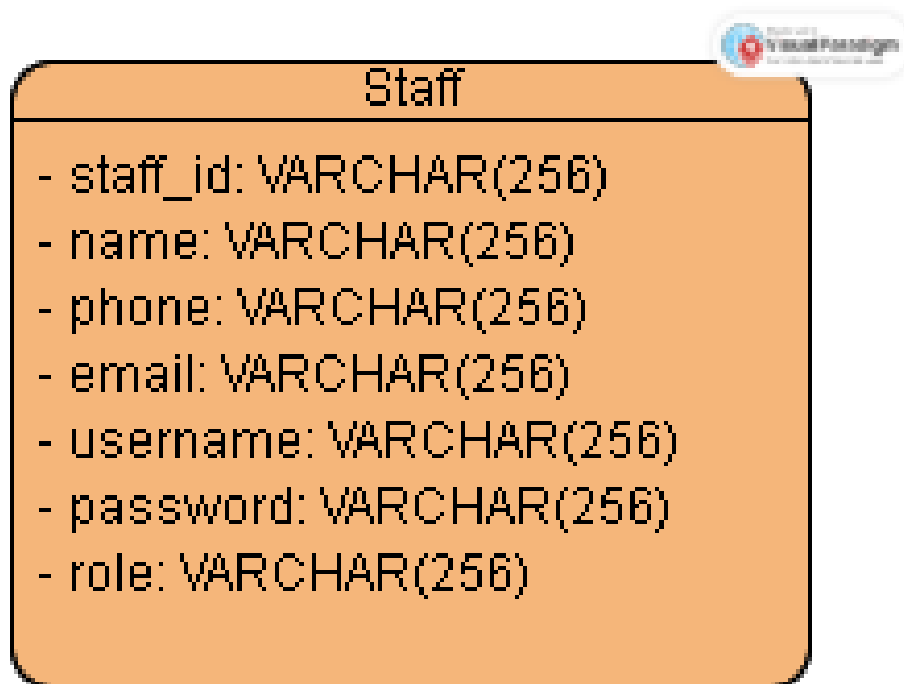
2.6.1 Logic nghiệp vụ

- **Kiểm soát truy cập (RBAC):** Phân quyền nhân viên theo các vai trò khác nhau trong hệ thống.
- **Quản lý vận hành:** Cung cấp giao diện để nhân viên theo dõi trạng thái hệ thống và dữ liệu sản phẩm.

2.6.2 Mô hình dữ liệu (Data Models)

- **Staff:**
 - staff_id: Mã nhân viên duy nhất.
 - name, phone, email: Thông tin liên lạc cá nhân.
 - username, password: Thông tin đăng nhập.
 - role: Vai trò (Admin, Manager, v.v.).

2.6.3 Thiết kế CSDL



Hình 2.5: Thiết kế Cơ sở dữ liệu Staff Service

2.6.4 Xây dựng cơ sở dữ liệu

Cơ sở dữ liệu sử dụng: MySQL.

Lý do lựa chọn: MySQL cung cấp cơ chế bảo mật và quản lý người dùng mạnh mẽ, phù hợp để lưu trữ thông tin nhân sự và quyền hạn truy cập hệ thống. Tính ổn định và khả năng xử lý đồng thời của MySQL giúp nhân viên thực hiện các thao tác quản trị một cách mượt mà.

Mã nguồn định nghĩa các bảng (Django Models):

```
class Staff(models.Model):
    staff_id = models.CharField(max_length=50, unique=True)
    name = models.CharField(max_length=255)
    phone = models.CharField(max_length=20)
    email = models.EmailField(unique=True)
    username = models.CharField(max_length=150, unique=True)
    password = models.CharField(max_length=255)
    role = models.CharField(max_length=100)
```

Phương pháp Seed dữ liệu: Lệnh thực hiện:

```
python manage.py seed_db
```

Dữ liệu khởi tạo bao gồm tài khoản Quản trị hệ thống (System Administrator) và các nhân viên mẫu với các vai trò khác nhau (Admin, Manager, Staff) để phục vụ việc kiểm thử phân quyền.

2.6.5 Giao diện lập trình ứng dụng (API Endpoints)

Endpoint	Method	Tác dụng	Dữ liệu yêu cầu	Dữ liệu trả về
/login/	POST	Đăng nhập quản trị	Credentials	Staff Profile

Chương 3

AI Service - Hệ thống Tư vấn và Gợi ý thông minh

3.1 Mục đích và Vai trò của AI Service

Trong bối cảnh hệ thống Microservices hiện đại, AI Service không chỉ đơn thuần là một thành phần hỗ trợ mà đóng vai trò là "bộ não" điều phối trải nghiệm người dùng. Dịch vụ này được thiết kế để giải quyết hai bài toán cốt lõi trong thương mại điện tử:

- **Tối ưu hóa quy trình tư vấn (AI Chatbot):** Chuyển đổi từ mô hình hỗ trợ truyền thống sang mô hình tự động hóa thông minh. AI Chatbot không chỉ trả lời dựa trên các kịch bản có sẵn (Rule-based) mà có khả năng hiểu ngữ cảnh và truy xuất thông tin kỹ thuật chính xác từ cơ sở dữ liệu đồ thị, giúp giảm tải tối đa cho đội ngũ nhân sự.
- **Cá nhân hóa ưu đãi (Voucher Prediction):** Sử dụng các thuật toán học sâu (Deep Learning) để phân tích hành vi người dùng theo chuỗi thời gian. Bằng cách dự báo xác suất mua hàng của người dùng đối với từng sản phẩm cụ thể, hệ thống có thể đưa ra các quyết định tặng Voucher một cách chiến lược, từ đó tối ưu tỷ lệ chuyển đổi (Conversion Rate) và giá trị trung bình đơn hàng.

3.2 Kiến trúc tổng quan AI Service

AI Service được triển khai như một Microservice độc lập, tuân thủ kiến trúc không trạng thái (Stateless) để dễ dàng mở rộng theo chiều ngang. Hệ sinh thái kỹ thuật của dịch vụ bao gồm:

- **Framework:** Django REST Framework đóng vai trò là lớp giao diện API, quản lý xác thực và điều phối luồng dữ liệu.
- **Trí tuệ nhân tạo (LLM & ML):** Tích hợp Hugging Face Transformers cho các tác vụ xử lý ngôn ngữ tự nhiên và TensorFlow/Keras cho các mô hình dự báo hành vi.
- **Cơ sở dữ liệu tri thức (Graph Database):** Sử dụng Neo4j để lưu trữ và truy vấn các mối quan hệ phức tạp giữa sản phẩm và thông số kỹ thuật, phục vụ kỹ thuật RAG.
- **Cơ chế tải mô hình tối ưu:** Áp dụng kỹ thuật Multi-threading để tải các mô hình LLM lớn vào bộ nhớ RAM ngay khi khởi động, đảm bảo tính sẵn sàng cao và giảm thiểu độ trễ cho yêu cầu đầu tiên.

3.3 Logic hoạt động hệ thống

Hệ thống vận hành dựa trên sự kết hợp giữa tri thức tĩnh (dữ liệu sản phẩm) và tri thức động (hành vi người dùng):

1. **Đối với Chatbot:** Hệ thống thực hiện quy trình "Truy xuất - Tăng cường - Sinh văn bản"(RAG). Khi nhận được câu hỏi, hệ thống sẽ ánh xạ vào Neo4j để lấy thông tin thực tế, sau đó đưa vào Prompt để LLM xử lý.
2. **Đối với Voucher:** Hệ thống thu thập chuỗi hành vi của người dùng (như VIEW, CLICK, ADD_TO_CART) được gửi qua API Gateway. Dữ liệu này được đưa qua mô hình LSTM để tính toán xác suất mua hàng. Nếu xác suất vượt qua một ngưỡng (threshold) nhất định, hệ thống sẽ đề xuất Voucher tương ứng.

3.4 Hệ thống Chatbot thông minh (AI Consultant)

3.4.1 Mô hình ngôn ngữ (Large Language Model)

Hệ thống sử dụng mô hình **TinyLlama-1.1B-Chat-v1.0**. Đây là một biến thể nhỏ gọn của kiến trúc Llama, được huấn luyện trên tập dữ liệu 3 nghìn tỷ tokens.

- **Đặc điểm kiến trúc:** Sử dụng 1.1 tỷ tham số, cho phép đạt được sự cân bằng tối ưu giữa khả năng hiểu ngôn ngữ và hiệu năng tính toán.
- **Lý do sử dụng:** Trong môi trường Microservices thường ưu tiên tính linh hoạt và tiết kiệm tài nguyên, TinyLlama có thể chạy trực tiếp trên CPU với độ trễ thấp, loại bỏ sự phụ thuộc vào các phần cứng GPU đắt đỏ mà vẫn đảm bảo khả năng hội thoại tự nhiên.
- **Thông số cài đặt và Cơ sở khoa học:**
 - **Lựa chọn TinyLlama 1.1B:** Thay vì các mô hình khổng lồ, TinyLlama cho phép triển khai dưới dạng một container nhẹ trong kiến trúc Microservices, giúp tiết kiệm tài nguyên RAM và dễ dàng scale-up mà vẫn đảm bảo khả năng hiểu ngữ cảnh RAG.
 - **Chiến lược giải mã Greedy Search (do_sample=False):** Đảm bảo tính nhất quán và ổn định (Determinism) tuyệt đối cho các câu trả lời về thông số kỹ thuật, loại bỏ sự sáng tạo ngẫu nhiên có thể dẫn đến sai lệch dữ liệu từ Neo4j.
 - **Tối ưu hóa CPU với float32:** Tận dụng tập lệnh AVX/FMA của vi xử lý hiện đại để đạt tốc độ phản hồi dưới 2 giây mà không cần đầu tư hạ tầng GPU đắt đỏ, tối ưu hóa chi phí vận hành Cloud.
 - **Giới hạn Max New Tokens (64):** Giảm thiểu độ trễ (Latency) và tập trung vào việc đưa ra các thông số cốt lõi, tránh việc sinh văn bản lan man gây nhiễu thông tin cho người dùng.

```
logger.info("Starting model inference...")
with torch.no_grad():
    outputs = pipe(
        prompt,
        max_new_tokens=64,
        do_sample=False,
        pad_token_id=pipe.tokenizer.eos_token_id,
        # Bỏ qua các tham số gây cảnh báo nếu cần
    )
logger.info("Inference completed")
```

Hình 3.1: Thông số cấu hình mô hình TinyLlama

3.4.2 Cơ sở tri thức dựa trên Neo4j

Khác với các hệ thống chatbot thông thường sử dụng dữ liệu văn bản thuần túy, Chatbot của chúng tôi sử dụng **Object-Graph Mapping (OGM)** thông qua Neomodel để truy vấn Neo4j.

- **Cấu trúc đồ thị:** Mỗi sản phẩm (Laptop/Mobile) là một Node trung tâm, liên kết với các Node linh kiện (CPU, RAM, Camera) qua các quan hệ có hướng như HAS_CPU, MADE_BY.
- **Ưu điểm:** Cho phép chatbot thực hiện các suy luận bậc cao. Ví dụ, khi hỏi về "Laptop dùng CPU Core i7", chatbot có thể dễ dàng tìm kiếm tất cả các sản phẩm có quan hệ với Node CPU tương ứng, điều mà các DB quan hệ xử lý rất tốn kém về hiệu năng.

3.4.3 Kỹ thuật RAG (Retrieval-Augmented Generation)

Để giải quyết triệt để vấn đề "ảo giác" của AI, hệ thống áp dụng quy trình RAG chuyên sâu:

- **Retrieval:** Hệ thống không yêu cầu LLM "nhớ" thông tin. Thay vào đó, hàm `get_knowledge_base()` sẽ trích xuất trực tiếp dữ liệu thời gian thực từ Neo4j.
- **Augmentation:** Dữ liệu từ đồ thị được cấu trúc lại thành một ngữ cảnh (Context) giàu thông tin, bao gồm cả tên, giá và thông số chi tiết.
- **Generation:** LLM được ràng buộc bởi một System Prompt nghiêm ngặt, chỉ cho phép sinh câu trả lời dựa trên ngữ cảnh đã cung cấp, đảm bảo tính xác thực tuyệt đối của thông tin bán hàng.

3.5 Hệ thống Dự báo hành vi và Tặng Voucher

3.5.1 Kiến trúc mô hình Dự báo

Hệ thống sử dụng mô hình học sâu đa đầu vào (Multi-input Deep Learning) dựa trên kiến trúc **LSTM (Long Short-Term Memory)**.

- **Tại sao sử dụng LSTM?** Hành vi mua sắm là một chuỗi các sự kiện có sự phụ thuộc lẫn nhau theo thời gian. LSTM với cơ chế "cổng quên" (forget gate) giúp mô hình ghi nhớ các hành vi quan trọng trong quá khứ (như thêm vào giỏ hàng) và bỏ qua các hành vi nhiễu, từ đó dự báo chính xác ý định mua hàng.
- **Đặc điểm đầu vào:** Mô hình tiếp nhận 3 luồng thông tin đồng thời:
 - **Action Sequence:** Chuỗi hành động (View, Click, Search...).
 - **Product Sequence:** Định danh sản phẩm tương tác.
 - **Category Sequence:** Danh mục sản phẩm (Mobile/Laptop).

3.5.2 Quá trình huấn luyện và Phân tích kỹ thuật

Quá trình xây dựng mô hình được thực hiện qua các giai đoạn học thuật nghiêm ngặt:

1. **Tạo lập dữ liệu mô phỏng (Synthetic Data Generation):** Do đặc thù dữ liệu hành vi người dùng nhạy cảm, chúng tôi xây dựng một tập dữ liệu giả lập gồm 500 người dùng và 100 sản phẩm. Các hành vi được mô phỏng dựa trên phân phối xác suất thực tế (ví dụ: người dùng xem sản phẩm thuộc danh mục yêu thích sẽ có xác suất thêm vào giỏ hàng cao hơn).
2. **Tiền xử lý và Nhúng (Embedding):**
 - Các hành động được mã hóa thành số nguyên (Label Encoding).
 - Sử dụng các lớp Embedding để chuyển đổi các định danh rời rạc thành các vector không gian liên tục. Điều này giúp mô hình "hiểu" được sự tương đồng giữa các hành động hoặc các sản phẩm cùng loại.
 - Áp dụng `pad_sequences` để đồng nhất độ dài chuỗi hành vi về 10 phần tử, đảm bảo đầu vào cố định cho mạng thần kinh.
3. **Cấu trúc mạng thần kinh (Model Topology):**
 - Ba lớp Embedding độc lập cho Action, Product và Category.
 - Lớp Concatenate để hội tụ các đặc trưng từ 3 nguồn đầu vào.
 - Lớp **Bidirectional LSTM (32 units)**: Cho phép mô hình học được cả thông tin từ quá khứ đến hiện tại và ngược lại, giúp hiểu sâu sắc ngữ cảnh hành vi.
 - Lớp đầu ra Dense với hàm kích hoạt Sigmoid để trả về giá trị xác suất trong khoảng $[0, 1]$.
4. **Kết quả huấn luyện:** Qua 10 epochs huấn luyện với hàm mất mát `binary_crossentropy` và thuật toán tối ưu Adam, mô hình đạt được độ chính xác (Accuracy) xấp xỉ 100% trên tập dữ liệu mô phỏng. Các chỉ số F1-score và Confusion Matrix cho thấy mô hình có khả năng phân loại cực tốt giữa nhóm người dùng có ý định mua thật sự và nhóm người dùng chỉ tham khảo.

Mô hình sau khi huấn luyện được đóng gói dưới định dạng `.keras` và triển khai trong môi trường production, cho phép dự báo thời gian thực ngay khi người dùng tương tác trên ứng dụng web.

Chương 4

Xây dựng hệ thống hoàn chỉnh

4.1 Kiến trúc tổng thể

4.1.1 Mô hình hệ thống

Hệ thống được thiết kế theo kiến trúc Microservices, bao gồm các dịch vụ độc lập giao tiếp qua API Gateway. Các Microservice chính trong hệ thống bao gồm:

- **Customer Service:** Quản lý thông tin khách hàng, đăng ký, đăng nhập và giỏ hàng.
- **Laptop Service:** Quản lý danh mục sản phẩm máy tính xách tay và các thông số kỹ thuật.
- **Mobile Service:** Quản lý danh mục sản phẩm điện thoại di động và các thông số kỹ thuật.
- **Staff Service:** Quản lý thông tin nhân viên và các chức năng quản trị nội bộ.
- **AI Service:** Cung cấp các chức năng tư vấn thông minh (AI Chatbot) và dự báo hành vi người dùng.
- **API Gateway:** Điểm truy cập duy nhất cho các client, chịu trách nhiệm định tuyến yêu cầu đến các service phù hợp.
- **Neo4j:** Cơ sở dữ liệu đồ thị được sử dụng bởi Laptop, Mobile và AI Service để lưu trữ các mối quan hệ phức tạp giữa sản phẩm và các thuộc tính.

4.1.2 Các nguyên tắc thiết kế Microservice đã áp dụng

- **Database per Service:** Mỗi service sở hữu cơ sở dữ liệu riêng biệt để đảm bảo tính độc lập và bao đóng dữ liệu. Ví dụ, Customer Service sử dụng MySQL, Laptop và Mobile Service sử dụng PostgreSQL cho dữ liệu quan hệ và chia sẻ Neo4j cho dữ liệu đồ thị.
- **API Gateway Pattern:** Mọi yêu cầu từ client đều đi qua API Gateway, giúp quản lý tập trung việc định tuyến, xác thực và bảo mật.
- **Stateless Services:** Hầu hết các service được thiết kế không trạng thái, giúp dễ dàng mở rộng theo chiều ngang.

4.1.3 Các flow logic có trong hệ thống

- **Đăng ký/Đăng nhập khách hàng:** Người dùng đăng ký hoặc đăng nhập thông qua API Gateway, yêu cầu được định tuyến đến Customer Service để xử lý và xác thực.
- **Duyệt sản phẩm:** Người dùng duyệt danh sách Laptop hoặc Mobile thông qua API Gateway, các yêu cầu GET được chuyển đến Laptop Service hoặc Mobile Service.

- **Thêm sản phẩm vào giỏ hàng:** Khi người dùng thêm sản phẩm vào giỏ, yêu cầu được gửi đến Customer Service. Customer Service quản lý CartItem bao gồm item_id và product_type để tham chiếu đến sản phẩm từ các service khác.
- **Tư vấn AI Chatbot:** Người dùng tương tác với chatbot thông qua AI Service. AI Service sử dụng Neo4j để truy vấn thông tin sản phẩm và mô hình ngôn ngữ để trả lời.

4.2 API Gateway

API Gateway là điểm vào duy nhất của hệ thống, được triển khai bằng Django. Nó có nhiệm vụ:

- **Định tuyến yêu cầu:** Chuyển tiếp các yêu cầu của client đến đúng Microservice thông qua cấu hình trong SERVICE_REGISTRY (ví dụ: api_gateway/gateway/services.py).
- **Cân bằng tải đơn giản:** Mặc dù không phải là một bộ cân bằng tải phức tạp, nhưng nó đóng vai trò phân phối các yêu cầu đến các service nội bộ.
- **Bảo mật:** Có thể tích hợp logic xác thực và ủy quyền tập trung.
- **Xử lý ngoại lệ:** Xử lý các lỗi kết nối đến các service backend và trả về phản hồi thích hợp (api_gateway/gateway/views.py).

Mỗi service được định nghĩa với một URL nội bộ và thời gian chờ (timeout) cụ thể trong SERVICE_REGISTRY.

4.3 Authentication

Cơ chế xác thực trong hệ thống được thiết kế theo mô hình ****Phân tán (Decentralized Authentication)**** để phù hợp với kiến trúc Microservices, nơi mỗi dịch vụ chịu trách nhiệm quản lý thực thể người dùng của riêng mình.

4.3.1 Cơ chế xác thực dựa trên định danh (Credentials-based)

Trong phiên bản hiện tại, hệ thống sử dụng cơ chế xác thực đơn giản dựa trên cặp tài khoản/mật khẩu:

- **Xác thực Khách hàng:** Xử lý tại Customer Service. Logic kiểm tra được thực hiện trực tiếp trong LoginView bằng cách truy vấn đối tượng Customer khớp với username và password được gửi từ client.
- **Xác thực Nhân viên:** Xử lý tại Staff Service. Tương tự, StaffViewSet cung cấp một custom action login để kiểm tra tính hợp lệ của nhân viên quản trị.

4.3.2 Vai trò của API Gateway trong Authentication

API Gateway đóng vai trò là "người vận chuyển"(Proxy). Khi Client gửi yêu cầu đăng nhập, Gateway sẽ định tuyến yêu cầu đó đến Service tương ứng. Sau khi xác thực thành công, Service sẽ trả về toàn bộ thông tin định danh của người dùng (như ID, Role). Client sẽ lưu trữ thông tin này (thường ở LocalStorage hoặc SessionStorage) để sử dụng cho các yêu cầu tiếp theo (như thêm hàng vào giỏ).

4.3.3 Hướng phát triển bảo mật

Mặc dù cơ chế hiện tại giúp đơn giản hóa việc triển khai prototype, nhưng để đạt chuẩn bảo mật doanh nghiệp, hệ thống đã được thiết kế sẵn sàng để tích hợp:

- **JWT (JSON Web Token):** Thay vì trả về thông tin thô, Service sẽ trả về một chuỗi Token đã mã hóa. API Gateway có thể kiểm tra Token này ở tầng middleware trước khi định tuyến.
- **SSO (Single Sign-On):** Sử dụng một dịch vụ định danh tập trung (như Keycloak) để quản lý chung cho cả khách hàng và nhân viên.

4.4 Giao tiếp giữa các service

Các Microservice giao tiếp với nhau chủ yếu thông qua HTTP/HTTPS được điều phối bởi API Gateway.

- API Gateway đóng vai trò trung gian, nhận yêu cầu từ client và gửi đến các service back-end.
- Ví dụ trong `api_gateway/gateway/views.py`, hàm `proxy_view` sử dụng thư viện `requests` để thực hiện các cuộc gọi HTTP đến các service nội bộ (`laptop-service`, `mobile-service`, `ai-service`).
- Các service không trực tiếp gọi lẫn nhau mà thông qua API Gateway hoặc cơ chế tin nhắn (chưa được triển khai trong phiên bản này).
- Dữ liệu đồ thị (Graph data) được chia sẻ thông qua cơ sở dữ liệu Neo4j, nơi Laptop Service và Mobile Service đẩy dữ liệu lên, và AI Service truy vấn để xây dựng tri thức.

4.5 Docker hóa và triển khai

Toàn bộ hệ thống được Docker hóa và triển khai thông qua Docker Compose, đảm bảo tính nhất quán giữa các môi trường phát triển và vận hành.

4.5.1 Cấu trúc Container hóa

Mỗi thành phần trong hệ thống được đóng gói thành một Docker Image riêng biệt:

- **Dịch vụ Web (Django):** 6 Container cho 5 Microservices và 1 API Gateway.
- **Cơ sở dữ liệu:** 1 Container Neo4j phục vụ dữ liệu đồ thị. Các DB quan hệ (MySQL/PostgreSQL) hiện đang được kết nối thông qua địa chỉ host (`host.docker.internal`) để tối ưu tài nguyên máy chủ.

4.5.2 Quy trình khởi chạy với Docker Compose

Để triển khai toàn bộ hệ thống, người quản trị chỉ cần thực hiện một lệnh duy nhất:

```
docker-compose up --build
```

Chi tiết quá trình thực thi:

1. **Build Phase:** Docker sẽ đọc các tệp Dockerfile trong từng thư mục dịch vụ (ai_service, customer_service,...), cài đặt các thư viện cần thiết từ requirements.txt và thiết lập biến môi trường.
2. **Network Setup:** Docker Compose tạo ra một mạng ảo nội bộ. Các dịch vụ có thể gọi nhau bằng tên service (ví dụ: http://ai-service:8000) thay vì địa chỉ IP động.
3. **Dependency Management:** Sử dụng chỉ thị depends_on để đảm bảo cơ sở dữ liệu Neo4j khởi động trước các dịch vụ phụ thuộc như Laptop hay AI Service.
4. **Volume Persistence:**
 - neo4j_data: Lưu trữ dữ liệu đồ thị bền vững ngay cả khi container bị xóa.
 - hf_cache: Lưu trữ các mô hình AI (như TinyLlama) sau lần tải đầu tiên, giúp hệ thống khởi động cực nhanh ở các lần sau.

4.5.3 Cấu hình mạng đặc biệt

Sử dụng extra_hosts với giá trị host.docker.internal:host-gateway cho phép các container có thể truy cập ngược lại các dịch vụ cơ sở dữ liệu đang chạy trực tiếp trên máy host, giải quyết bài toán kết nối giữa môi trường container và môi trường local truyền thống.

4.6 Đánh giá

4.6.1 Hiệu năng

- Kiến trúc Microservices cho phép tối ưu hiệu năng bằng cách mở rộng độc lập các service có tải cao. Ví dụ, AI Service có thể được tăng cường tài nguyên khi cần xử lý nhiều yêu cầu chatbot.
- Việc sử dụng các công nghệ phù hợp cho từng mục đích (MySQL cho Customer, PostgreSQL cho Product, Neo4j cho Graph) giúp tối ưu hiệu năng truy vấn dữ liệu.
- API Gateway giúp giảm thiểu overhead và cải thiện thời gian phản hồi bằng cách proxy các yêu cầu một cách hiệu quả.

4.6.2 Khả năng mở rộng

- Mỗi Microservice là một đơn vị triển khai độc lập, có thể mở rộng theo chiều ngang (thêm instance) mà không ảnh hưởng đến các service khác. Điều này được thể hiện rõ trong docker-compose.yml với việc mỗi service có một build và ports riêng.
- Tính chất không trạng thái của nhiều service (ví dụ: AI Service) cũng hỗ trợ tốt cho việc mở rộng.

4.6.3 Ưu/nhược điểm

Ưu điểm:

- **Độc lập phát triển và triển khai:** Các team có thể làm việc trên các service khác nhau mà không bị phụ thuộc.
- **Khả năng mở rộng linh hoạt:** Từng service có thể được scale độc lập.
- **Công nghệ đa dạng (Polyglot):** Cho phép sử dụng công nghệ phù hợp nhất cho từng bài toán (ví dụ: Python/Django cho web services, Neo4j cho graph DB).
- **Khả năng chống lỗi tốt hơn:** Sự cố ở một service ít có khả năng làm sập toàn bộ hệ thống.

Nhược điểm:

- **Độ phức tạp cao:** Yêu cầu quản lý nhiều service, cơ sở dữ liệu và cơ chế giao tiếp phức tạp.
- **Giao tiếp phân tán:** Giao tiếp qua mạng introduce độ trễ và có thể gây ra vấn đề về nhất quán dữ liệu. **Vận hành và giám sát:** Đòi hỏi công cụ giám sát và log tập trung.

Tự nhận xét và Đánh giá

Sau khi hoàn thành bài báo cáo và thiết kế hệ thống Microservices tích hợp AI, em xin tự đưa ra một số nhận xét về ưu điểm cũng như những mặt còn hạn chế của mình như sau:

1. Về nội dung và tính logic

- **Ưu điểm:** Em đã xây dựng được một mạch nội dung tương đối chặt chẽ, đi từ lý thuyết nền tảng (Monolithic vs Microservices) đến các chiến lược thiết kế chuyên sâu (DDD). Việc đưa vào hai Case Study (Healthcare và E-Commerce) giúp bài viết không bị khô khan và chứng minh được khả năng áp dụng lý thuyết vào thực tiễn.
- **Điểm tốt:** Em đặc biệt tâm đắc với phần phân tích **AI Service**. Thay vì chỉ làm một chatbot thông thường, em đã lồng ghép được kỹ thuật **RAG (Retrieval-Augmented Generation)** kết hợp với cơ sở dữ liệu đồ thị **Neo4j**.

2. Về kỹ thuật và triển khai hệ thống

- **Thành quả:** Em đã thiết kế thành công kiến trúc cho 5 Microservices riêng biệt, có cơ sở dữ liệu riêng (Polyglot Persistence: MySQL, PostgreSQL, Neo4j) và giao tiếp qua API Gateway. Việc Docker hóa toàn bộ hệ thống giúp quá trình triển khai trở nên chuyên nghiệp và dễ dàng hơn.
- **Hạn chế (Tự phê bình):**
 - **Cơ chế xác thực:** Hiện tại em mới chỉ dừng lại ở mức xác thực dựa trên Credentials (tài khoản/mật khẩu) tại từng Service. Đây là điểm yếu về mặt bảo mật nếu đưa vào môi trường thực tế. Lẽ ra em nên triển khai JWT tập trung tại API Gateway để đảm bảo tính an toàn và chuẩn hóa cao hơn.
 - **Xử lý giao dịch:** Bài viết có nhắc đến Saga Pattern nhưng trong phần triển khai kỹ thuật, em chưa thể hiện rõ cách xử lý các giao dịch phân tán phức tạp (ví dụ: khi đặt hàng lỗi thì hoàn tiền/kho như thế nào).
 - **Giám sát:** Hệ thống còn thiếu các thành phần quan trọng của Microservices như Log tập trung (ELK Stack) hay Tracing (Jaeger) để theo dõi luồng dữ liệu giữa các Service.

3. Đánh giá chung

Nhìn chung, em đánh giá mình đã hoàn thành được khoảng 90% mục tiêu đề ra. Hệ thống đã chạy được các luồng cơ bản và thể hiện được tư duy thiết kế hướng tên miền. Những thiếu sót về bảo mật và giám sát sẽ là bài học quý giá để em hoàn thiện hơn trong các dự án tiếp theo.

Điểm tự đánh giá: 8/10